

第 9 章：并发控制

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

教学内容¹

- ① 9.1 并发控制
- ② 9.2 基于锁的并发控制
 - 9.2.1 锁
 - 9.2.2 两阶段封锁协议
 - 9.2.3 级联中止
 - 9.2.4 死锁
 - 9.2.5 多粒度锁
 - 9.2.6 幻读异常
- ③ 9.3 基于时间戳的并发控制
 - 9.3.1 基于时间戳的并发控制的基本概念
 - 9.3.2 基本时间戳定序协议
 - 9.3.2 乐观并发控制协议
- ④ 9.4 多版本并发控制
 - 9.4.1 多版本并发控制的基本概念
 - 9.4.2 多版本并发控制下的操作

¹课件更新于 2026 年 3 月 13 日

9.1 并发控制

- 对并发事务实施正确的（运行时）调度
- 无需预先确定整个（静态）调度

并发控制协议的分类（一）

按使用场景分类

悲观并发控制（Pessimistic Concurrency Control）

- 假定冲突很多
- 不允许任何冲突发生

乐观并发控制（Optimistic Concurrency Control）

- 假定冲突很少
- 冲突发生了，再去处理

并发控制协议的分类（二）

按实现方法分为：

- 基于锁的并发控制（Lock-based Concurrency Control）
- 时间戳定序的并发控制（Timestamp-based Concurrency Control）
- 多版本并发控制（Multi-Version Concurrency Control, MVCC）

9.2 基于锁的并发控制

9.2.1 锁

锁 (Lock)

- 锁刻画了数据对象被事务访问的状态
- 锁用来保护被竞争访问的数据对象

封锁协议 (Locking Protocol)

- 事务 T 读或写对象 A ，必须遵守如下规则：
- ❶ T 只有获得了 A 的锁，才能读或写 A
 - ❷ 如果 T 请求了 A 的锁，但并未获得，则 T 开始等待，直至获得 A 的锁为止
 - ❸ 如果 T 已经获得了 A 的锁，则在 T 完成对 A 的操作后， T 必须释放 A 的锁

锁的使用

- LOCK(A)：请求对数据对象 A 加锁
- UNLOCK(A)：释放数据对象 A 上的锁

例 (锁的使用)		
事务 T_1	事务 T_2	调度器的动作
LOCK(A)		将 A 上的锁授予 T_1
$r(A)$		
	LOCK(A)	
$w(A)$		
$r(A)$		
UNLOCK(A)		
	$r(A)$	
	$w(A)$	
	UNLOCK(A)	

锁的使用

- LOCK(A)：请求对数据对象 A 加锁
- UNLOCK(A)：释放数据对象 A 上的锁

例 (锁的使用)		
事务 T_1	事务 T_2	调度器的动作
LOCK(A)		将 A 上的锁授予 T_1
$r(A)$		
	LOCK(A)	拒绝
$w(A)$		
$r(A)$		
UNLOCK(A)		
	$r(A)$	
	$w(A)$	
	UNLOCK(A)	

锁的使用

- LOCK(A): 请求对数据对象 A 加锁
- UNLOCK(A): 释放数据对象 A 上的锁

例 (锁的使用)

事务 T_1	事务 T_2	调度器的动作
LOCK(A)		将 A 上的锁授予 T_1
r(A)		
	LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	
	w(A)	
	UNLOCK(A)	

锁的使用

- LOCK(A): 请求对数据对象 A 加锁
- UNLOCK(A): 释放数据对象 A 上的锁

例 (锁的使用)

事务 T_1	事务 T_2	调度器的动作
LOCK(A)		将 A 上的锁授予 T_1
r(A)		
	LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	将 A 上的锁授予 T_2
	w(A)	
	UNLOCK(A)	

锁的使用

- LOCK(A): 请求对数据对象 A 加锁
- UNLOCK(A): 释放数据对象 A 上的锁

例 (锁的使用)

事务 T_1	事务 T_2	调度器的动作
LOCK(A)		将 A 上的锁授予 T_1
r(A)		
	LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	将 A 上的锁授予 T_2
	w(A)	
	UNLOCK(A)	释放 A 上的锁

锁的类型

定义 (共享锁 (Shared Lock) / S 锁 (S-lock))

- 事务 T 只有获得了对象 A 的共享锁, 才能读 A

定义 (互斥锁 (Exclusive Lock) / X 锁 (X-lock))

- 事务 T 只有获得了对象 A 的互斥锁, 才能写 A
- T 获得了 A 的互斥锁后, 亦可读 A

锁的相容性 (Compatibility)

- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 还可以对 A 加 X 锁
- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 不可以对 A 加 X 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 S 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 X 锁; 否则, 可能产生写-写冲突

定义 (锁相容矩阵 (Lock Compatibility Matrix))

		请求加锁的类型	
		共享锁	互斥锁
已加锁的类型	共享锁	Yes	
	互斥锁		

锁的相容性 (Compatibility)

- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 还可以对 A 加 X 锁
- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 不可以对 A 加 X 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 S 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 X 锁; 否则, 可能产生写-写冲突

定义 (锁相容矩阵 (Lock Compatibility Matrix))

		请求加锁的类型	
		共享锁	互斥锁
已加锁的类型	共享锁	Yes	No
	互斥锁		

锁的相容性 (Compatibility)

- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 还可以对 A 加 X 锁
- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 不可以对 A 加 X 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 S 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 X 锁; 否则, 可能产生写-写冲突

定义 (锁相容矩阵 (Lock Compatibility Matrix))

		请求加锁的类型	
		共享锁	互斥锁
已加锁的类型	共享锁	Yes	No
	互斥锁	No	

锁的相容性 (Compatibility)

- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 还可以对 A 加 X 锁
- 如果对象 A 上有事务 T_i 加的 S 锁, 则事务 T_j 不可以对 A 加 X 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 S 锁; 否则, 可能产生读-写冲突或写-读冲突
- 如果对象 A 上有事务 T_i 加的 X 锁, 则事务 T_j 对 A 不可以对 A 加 X 锁; 否则, 可能产生写-写冲突

定义 (锁相容矩阵 (Lock Compatibility Matrix))

		请求加锁的类型	
		共享锁	互斥锁
已加锁的类型	共享锁	Yes	No
	互斥锁	No	No

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	
w(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	A 上的 X 锁授予 T_2
	w(A)	
S-LOCK(A)		
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	A 上的 X 锁授予 T_2
	w(A)	
S-LOCK(A)		拒绝
	UNLOCK(A)	
r(A)		
UNLOCK(A)		

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	A 上的 X 锁授予 T_2
	w(A)	
S-LOCK(A)		拒绝
	UNLOCK(A)	释放 A 上的 X 锁
r(A)		
UNLOCK(A)		

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	A 上的 X 锁授予 T_2
	w(A)	
S-LOCK(A)		拒绝
	UNLOCK(A)	释放 A 上的 X 锁
r(A)		A 上的 S 锁授予 T_1
UNLOCK(A)		

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	A 上的 X 锁授予 T_2
	w(A)	
S-LOCK(A)		拒绝
	UNLOCK(A)	释放 A 上的 X 锁
r(A)		A 上的 S 锁授予 T_1
UNLOCK(A)		释放 A 上的 S 锁

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	A 上的 X 锁授予 T_2 是冲突可串行化调度吗?
	w(A)	
S-LOCK(A)		拒绝
	UNLOCK(A)	释放 A 上的 X 锁
r(A)		A 上的 S 锁授予 T_1
UNLOCK(A)		释放 A 上的 S 锁

邹兆年 (CS@HIT)

第 9 章: 并发控制

2026 春

14 / 128

共享锁与互斥锁的使用

- S-LOCK(A): 请求对 A 加共享锁
- X-LOCK(A): 请求对 A 加互斥锁

例 (共享锁与互斥锁的使用)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
UNLOCK(A)		释放 A 上的 X 锁
	r(A)	A 上的 X 锁授予 T_2 尽管使用了锁, 却不是冲突可串行化
	w(A)	
S-LOCK(A)		拒绝
	UNLOCK(A)	释放 A 上的 X 锁
r(A)		A 上的 S 锁授予 T_1
UNLOCK(A)		释放 A 上的 S 锁

邹兆年 (CS@HIT)

第 9 章: 并发控制

2026 春

14 / 128

9.2.2 两阶段封锁协议

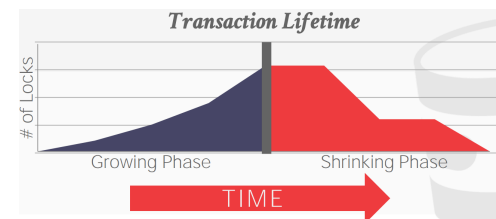
两阶段封锁协议 (Two-Phase Locking, 2PL)

增长阶段 (Growing Phase)

- 事务向锁管理器 (Lock Manager) 请求需要的锁

收缩阶段 (Shrinking Phase)

- 事务释放它获得的锁, 但不能再请求加锁



2

²来源: Andy Pavlo, CMU 15-445

邹兆年 (CS@HIT)

第 9 章: 并发控制

2026 春

15 / 128

邹兆年 (CS@HIT)

第 9 章: 并发控制

2026 春

16 / 128

基于 2PL 的并发控制

例 (2PL)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	
w(A)		
r(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
	UNLOCK(A)	

基于 2PL 的并发控制

例 (2PL)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
	UNLOCK(A)	

基于 2PL 的并发控制

例 (2PL)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	
	w(A)	
	UNLOCK(A)	

基于 2PL 的并发控制

例 (2PL)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	将 A 上的 X 锁授予 T_2
	w(A)	
	UNLOCK(A)	

基于 2PL 的并发控制

例 (2PL)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	将 A 上的 X 锁授予 T_2
	w(A)	
	UNLOCK(A)	释放 A 上的 X 锁

基于 2PL 的并发控制

例 (2PL)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	将 A 上的 X 锁授予 T_2
	w(A)	
	UNLOCK(A)	释放 A 上的 X 锁

是冲突可串行化调度吗?

基于 2PL 的并发控制

例 (2PL)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	X-LOCK(A)	拒绝
w(A)		
r(A)		
UNLOCK(A)		释放 A 上的锁
	r(A)	将 A 上的 X 锁授予 T_2
	w(A)	
	UNLOCK(A)	释放 A 上的 X 锁

不仅是冲突可串行化调度，还是串行调度（没有并发执行）

2PL 的优点

定理 (2PL 冲突可串行化)

2PL 保证冲突可串行化

2PL 的缺点

- 存在级联中止 (Cascading Abort)问题
- 可能导致死锁 (Deadlock)

9.2.3 级联中止

级联中止 (Cascading Abort)

一个事务中止导致其他事务必须中止

例 (级联中止)

事务 T ₁	事务 T ₂
X-LOCK(A)	
X-LOCK(B)	
r(A)	X-LOCK(A)
w(A)	
UNLOCK(A)	
	r(A)
	w(A)
r(B)	
w(B)	
ABORT	
	r(A)

- 2PL 允许该调度
- 该调度存在脏读异常 (Dirty Read)
- T₁ 中止时, T₂ 必须级联中止, 否则 T₁ 修改过的数据将通过 T₂ 泄露给外界

级联中止 (Cascading Abort)

一个事务中止导致其他事务必须中止

例 (级联中止)

事务 T ₁	事务 T ₂
X-LOCK(A)	
X-LOCK(B)	
r(A)	X-LOCK(A)
w(A)	
UNLOCK(A)	
	r(A)
	w(A)
r(B)	
w(B)	
ABORT	
	r(A)

- 2PL 允许该调度
- 该调度存在脏读异常 (Dirty Read)
- T₁ 中止时, T₂ 必须级联中止, 否则 T₁ 修改过的数据将通过 T₂ 泄露给外界

级联中止 (Cascading Abort)

一个事务中止导致其他事务必须中止

例 (级联中止)

事务 T_1	事务 T_2	
X-LOCK(A)		
X-LOCK(B)		
r(A)	X-LOCK(A)	
w(A)		
UNLOCK(A)		
	r(A)	
	w(A)	
r(B)		
w(B)		
ABORT		
	r(A)	

- 2PL 允许该调度
- 该调度存在脏读异常 (Dirty Read)
- T_1 中止时, T_2 必须级联中止, 否则 T_1 修改过的数据将通过 T_2 泄露给外界

严格调度 (Strict Schedule)

定义 (严格调度)

如果一个调度中任意事务结束前由它写入的数据对象的值都没有被其他事务读过或修改过, 则该调度称作严格调度

- 严格调度显然不会产生级联中止

强严格两阶段封锁协议 (Strong Strict 2PL, SS2PL/Rigorous 2PL)

增长阶段 (Growing Phase)

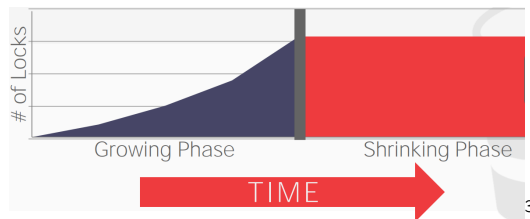
- 与 2PL 的增长阶段相同

萎缩阶段 (Shrinking Phase)

- 事务结束时, 才释放它获得的全部锁

定理 (SS2PL)

SS2PL 保证生成严格调度



³来源: Andy Pavlo, CMU 15-445

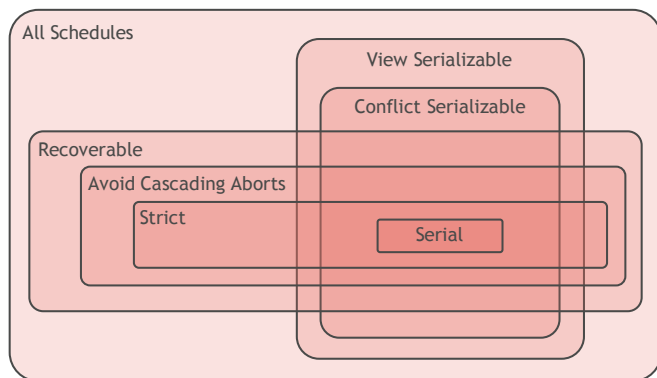
SS2PL

例 (SS2PL)

事务 T_1	事务 T_2
X-LOCK(A)	
X-LOCK(B)	
r(A)	X-LOCK(A)
w(A)	
r(B)	
w(B)	
UNLOCK(A)	
UNLOCK(B)	
ABORT	
	r(A)
	w(A)
	r(A)

是严格调度吗?

调度的分类



9.2.4 死锁

死锁 (Deadlock)

定义 (死锁)

如果一组事务中每个事务都在等待其他事务释放锁，则这组事务形成死锁

例 (死锁)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	S-LOCK(B)	
	r(B)	
	S-LOCK(A)	
w(A)		
X-LOCK(B)		
⋮	⋮	

死锁 (Deadlock)

定义 (死锁)

如果一组事务中每个事务都在等待其他事务释放锁，则这组事务形成死锁

例 (死锁)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	S-LOCK(B)	将 B 上的 S 锁授予 T_2
	r(B)	
	S-LOCK(A)	
w(A)		
X-LOCK(B)		
⋮	⋮	

死锁 (Deadlock)

定义 (死锁)

如果一组事务中每个事务都在等待其他事务释放锁，则这组事务形成死锁

例 (死锁)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	S-LOCK(B)	将 B 上的 S 锁授予 T_2
	r(B)	
	S-LOCK(A)	拒绝!
w(A)		
X-LOCK(B)		
⋮	⋮	

死锁 (Deadlock)

定义 (死锁)

如果一组事务中每个事务都在等待其他事务释放锁，则这组事务形成死锁

例 (死锁)

事务 T_1	事务 T_2	调度器的动作
X-LOCK(A)		将 A 上的 X 锁授予 T_1
r(A)		
	S-LOCK(B)	将 B 上的 S 锁授予 T_2
	r(B)	
	S-LOCK(A)	拒绝!
w(A)		
X-LOCK(B)		拒绝!
⋮	⋮	

形成死锁!

死锁的处理

措施 1: 死锁检测 (Deadlock Detection)

- DBMS 检测死锁是否发生
- 如果发生了死锁，则采取办法解除死锁

措施 2: 死锁预防 (Deadlock Prevention)

- 预防死锁的发生

死锁检测 (Deadlock Detection)

方法 1: 超时 (Timeout 检测法)

- 如果在给定的时间内没有任何事务执行，则认为死锁发生

方法 2: 等待图 (Waits-for Graph) 检测法

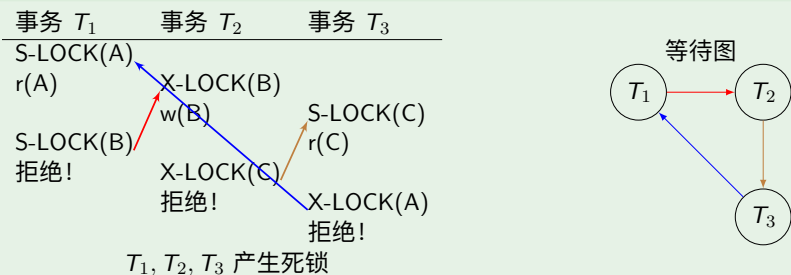
- DBMS 用等待图表示事务关于锁的等待关系
- DBMS 定期检查等待图中是否存在环 (Cycle)
- 如果等待图中有环，则表示发生了死锁

等待图 (Waits-for Graph)

定义 (等待图)

- 顶点代表事务
- 如果事务 T_i 正在等待事务 T_j 释放锁, 则存在从 T_i 到 T_j 的有向边

例 (等待图)

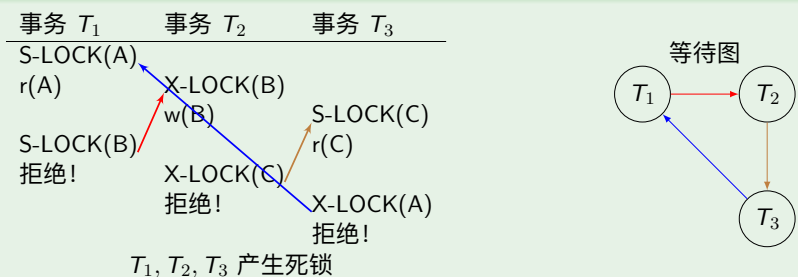


等待图 (Waits-for Graph)

定理

事务产生死锁当且仅当等待图中有环

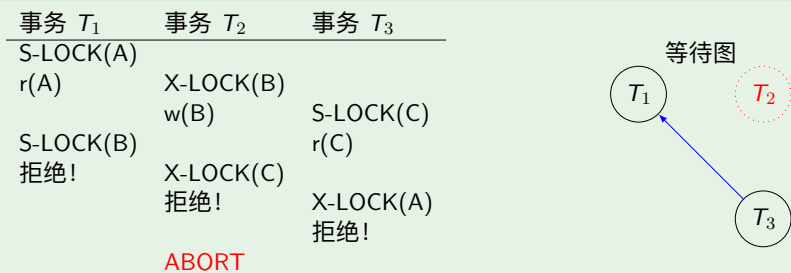
例 (等待图)



死锁解除 (Deadlock Resolution)

从等待图的环中选择一个事务作为“牺牲品”, 中止该事务

例 (死锁解除)



死锁解除

选择“牺牲品”时需要考虑多种因素

- 事务的年龄/启动时间
- 事务的进度 (已执行的查询数量)
- 事务获得的锁的数量
- 需要级联中止的事务数量
- 还要考虑事务“被牺牲”的次数, 防止事务“饿死” (Starvation)

死锁预防 (Deadlock Prevention)

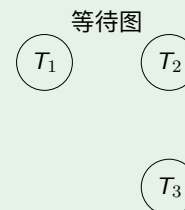
- No-Wait 死锁预防策略
- 基于优先级的死锁预防策略
- 基于统一请求加锁顺序的死锁预防策略

No-Wait 死锁预防

事务 T 请求对对象 X 加锁, 如果被拒绝, 则 T 中止, 然后重启为新事务

例 (No-wait 死锁预防)

事务 T_1	事务 T_2	事务 T_3
S-LOCK(A) r(A)		
	X-LOCK(B) w(B)	
		S-LOCK(C) r(C)
S-LOCK(B) ABORT		
	X-LOCK(C) ABORT	
		X-LOCK(A) ABORT



基于优先级的死锁预防

定义 (事务的优先级)

当事务启动时, DBMS 为事务分配唯一且固定的优先级 (Priority)

- 事务启动得越早, 优先级越高

当事务 T_i 请求事务 T_j 拥有的锁而无法获得时, DBMS 根据 T_i 和 T_j 的优先级裁决如何处理 T_i 和 T_j , 可采用下面两条规则之一来预防

- Wait-Die 规则
- Wound-Wait 规则

使用上述规则来处理加锁请求, 则不会产生死锁

Wait-Die 规则—“谦让”

定义 (Wait-Die 规则)

- 如果 T_i 比 T_j 优先级高, 则 T_i 等待
- 否则, T_i 中止; T_i 重启后, 优先级保持不变 (为什么?)

例 (Wait-Die 规则)

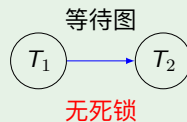
设 T_1 比 T_2 优先级高

T_1	T_2	Wait-Die
X-LOCK(A)	X-LOCK(A)	T_1 等待
T_1	T_2	Wait-Die
X-LOCK(A)	X-LOCK(A)	T_2 中止

Wait-Die 规则—“谦让”

例 (Wait-Die 规则)

T_1	T_2	T_3
S-LOCK(A) r(A)		
	X-LOCK(B) w(B)	
		S-LOCK(C) r(C)
S-LOCK(B) 等待		
	X-LOCK(C) 等待	
		X-LOCK(A) 中止
		获得 C 的 X 锁



Wound-Wait 规则—“霸道”

定义 (Wound-Wait 规则)

- 如果 T_i 比 T_j 优先级高，则 T_j 中止； T_j 重启后，优先级保持不变
- 否则， T_i 等待

例 (Wound-Wait 规则)

设 T_1 比 T_2 优先级高

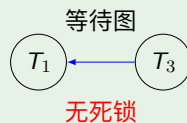
T_1	T_2	Wound-Wait
	X-LOCK(A)	
X-LOCK(A)		T_2 中止

T_1	T_2	Wound-Wait
X-LOCK(A)		
	X-LOCK(A)	T_2 等待

Wound-Wait 规则—“霸道”

例 (Wound-Wait 规则)

T_1	T_2	T_3
S-LOCK(A) r(A)		
	X-LOCK(B) w(B)	
		S-LOCK(C) r(C)
S-LOCK(B) 中止 T_2		
	ABORT	
		X-LOCK(A) 等待



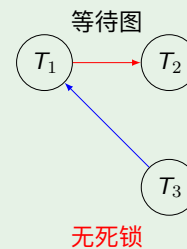
基于统一请求加锁顺序的死锁预防

规定数据对象的统一顺序，每个事务必须按顺序对数据对象请求加锁

例 (基于统一请求加锁顺序的死锁预防)

统一的顺序：A, B, C

T_1	T_2	T_3
S-LOCK(A) r(A)		
	X-LOCK(B) w(B)	
S-LOCK(B), 等待		X-LOCK(A), 等待
	X-LOCK(C) w(C) UNLOCK(B) UNLOCK(C)	
r(B) UNLOCK(A) UNLOCK(B)		
		w(A) S-LOCK(C) r(C)



9.2.5 多粒度锁

锁的效率问题

锁越多，管理锁开销越大

- 一个事务访问 1 亿条元组，需要加 1 亿把锁

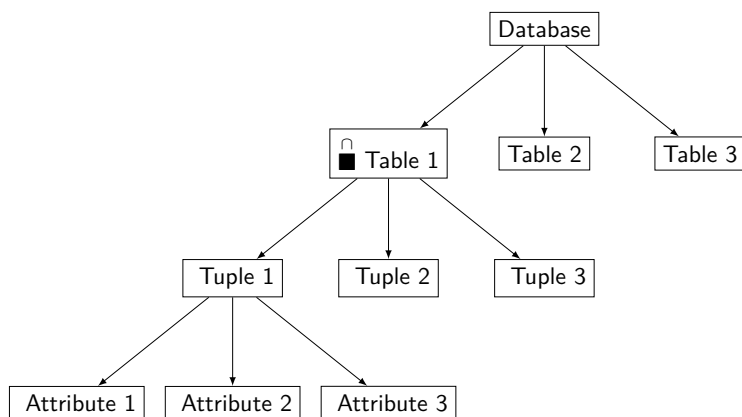
在不合适的粒度上加锁会降低事务并发度

- 一个事务只需访问 1 条元组，却在整个关系上加锁

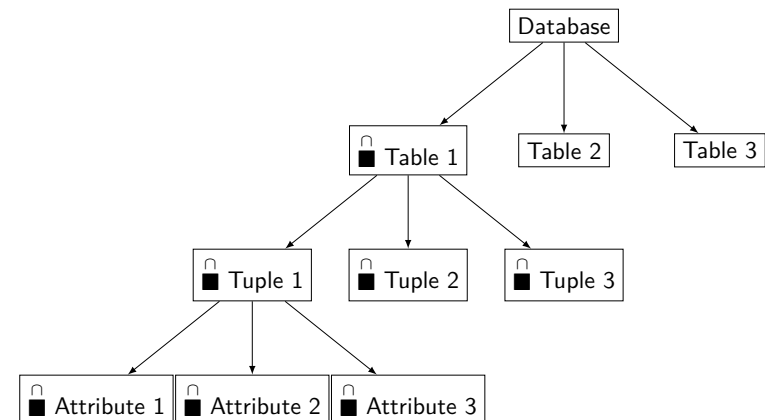
提高锁管理的效率

- 尽可能少加锁
- 在合适的粒度上加锁

锁的粒度 (Lock Granularity)



锁的粒度 (Lock Granularity)



意向锁 (Intension Lock)

定义 (意向共享锁 (Intension-Shared Lock) / IS 锁 (IS-Lock))

对一个对象加 IS 锁表示要对该对象的某个 (些) 后裔加 S 锁

定义 (意向互斥锁 (Intension-Exclusive Lock) / IX 锁 (IX-Lock))

对一个对象加 IX 锁表示要对该对象的某个 (些) 后裔加 X 锁或 S 锁

定义 (共享意向互斥锁 (Shared Intension-Exclusive Lock) / SIX 锁 (SIX-Lock))

对一个对象加 SIX 锁表示要对该对象及其所有后裔加 S 锁, 并且对该对象的某个 (些) 后裔加 X 锁

- SIX = S + IX

锁相容矩阵 (Lock Compatibility Matrix)

		请求锁类型					
		IS	IX	S	SIX	X	
拥有锁类型	IS	Yes	Yes	Yes	Yes	No	
	IX	Yes	Yes	No	No	No	
	S	Yes	No	Yes	No	No	
	SIX	Yes	No	No	No	No	
	X	No	No	No	No	No	

多粒度封锁协议 (Multi-Granularity Locking)

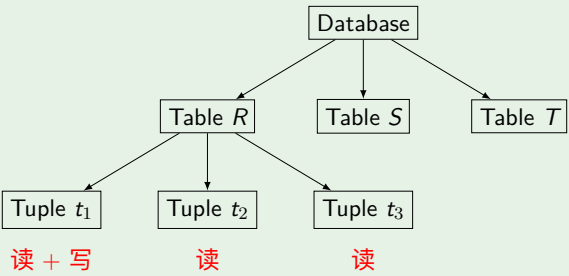
任何事务都要服从下列规则:

- 从最高级别对象开始加锁, 加锁过程自顶向下
- 对一个对象加 IS 或 S 锁之前, 必须先获得其父亲对象的 IS 锁
- 事务对一个对象加 IX、SIX 或 X 锁之前, 必须先获得其父亲对象的 IX 锁
- 解锁过程自底向上

多粒度封锁协议

例 (多粒度锁协议)

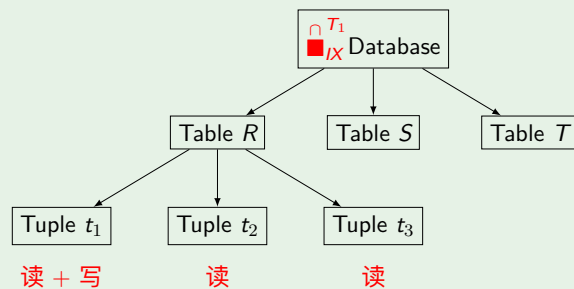
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

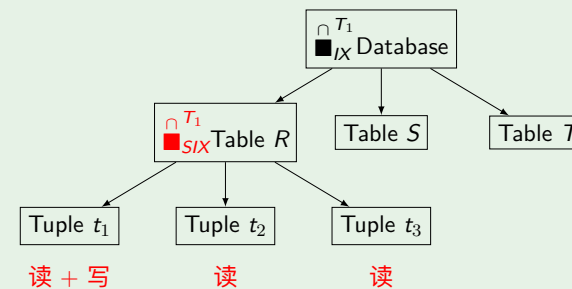
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

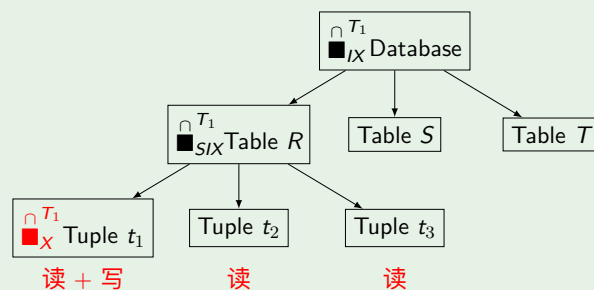
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

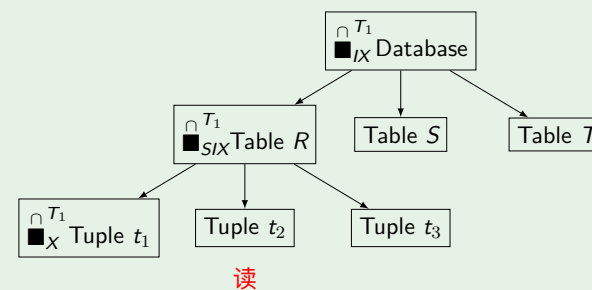
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

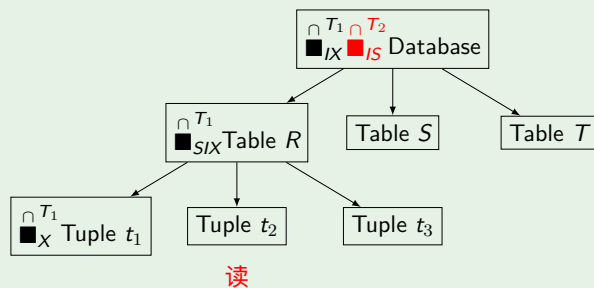
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

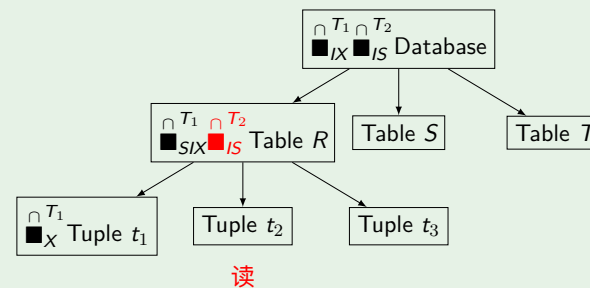
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

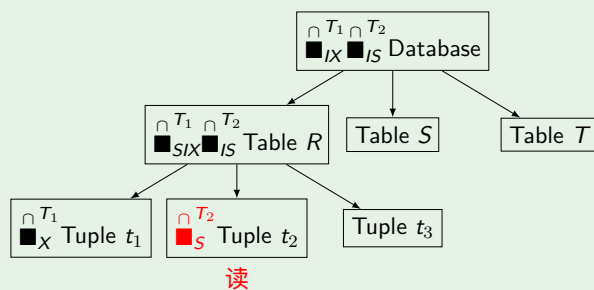
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

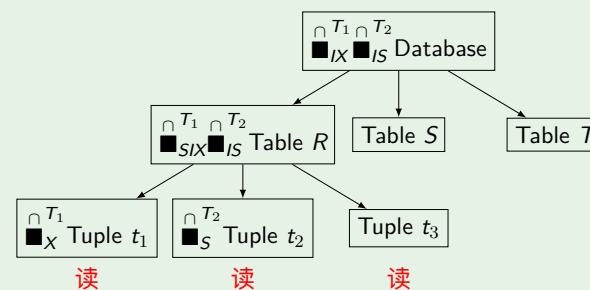
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度封锁协议)

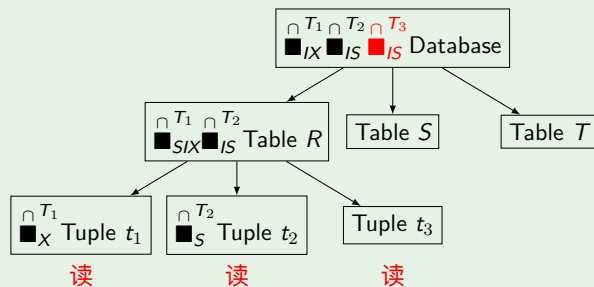
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度锁协议)

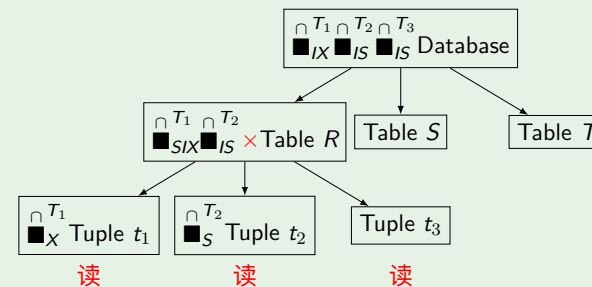
- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R



多粒度封锁协议

例 (多粒度锁协议)

- 事务 T_1 扫描关系 R 并修改元组 t_1
- 事务 T_2 读关系 R 的元组 t_2
- 事务 T_3 扫描关系 R : 对 R 加 S 锁被拒, T_3 等待



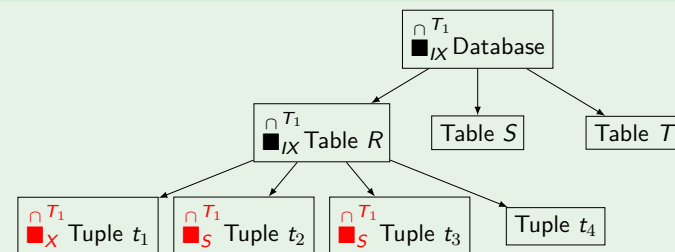
锁升级 (Lock Escalation)

如果一个事务已经请求了大量低级别对象上的锁, 则 DBMS 会动态地将这些锁升级为上一级别对象上的锁

- 减少锁的数量
- 选择合适的锁粒度

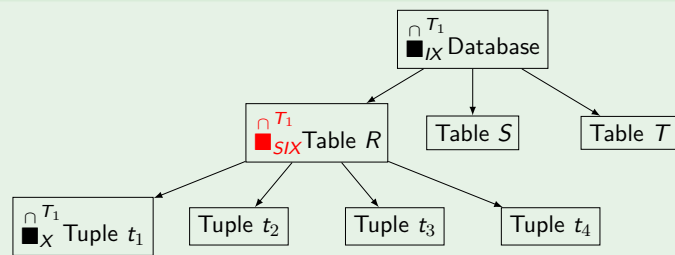
锁升级 (Lock Escalation)

例 (锁升级)



锁升级 (Lock Escalation)

例 (锁升级)



9.2.6 幻读异常

动态数据库 (Dynamic Database)

- 前面假设事务只执行读操作和更新操作
- 冲突可串行化调度只在固定的数据库上保证
- 如果事务还执行插入或删除操作，则可能出现幻读 (Phantom) 异常

幻读 (Phantom)

例 (幻读)

事务 T_1	事务 T_2	T_1 的结果
BEGIN	BEGIN	
SELECT MAX(val) FROM R		200
WHERE id > 1;		
	INSERT INTO t	
	VALUES (3, 300);	
	COMMIT;	
SELECT MAX(val) FROM R		300
WHERE id > 1;		
COMMIT;		

原因: T_1 只能锁定 R 中 id > 1 的元组, 无法对尚不存在的元组加锁

R

id	val
1	100
2	200

谓词锁 (Predicate Lock)

不对对象加锁，而是对查询谓词加锁

例 (谓词锁)

事务 T_1	事务 T_2	调度器的动作
<code>LOCK('id > 1')</code>		将锁授予 T_1
<code>SELECT MAX(val) FROM t</code> <code>WHERE id > 1;</code>		
	<code>LOCK('id = 3')</code>	拒绝
<code>SELECT MAX(val) FROM t</code> <code>WHERE id > 1;</code>		
<code>UNLOCK('id > 1')</code>		释放锁
<code>COMMIT</code>		
	<code>INSERT INTO t</code> <code>VALUES (3, 300);</code> <code>UNLOCK('id = 3');</code> <code>COMMIT</code>	将锁授予 T_2

谓词锁 (Predicate Lock)

谓词锁想法不切实际

- 谓词的数量比对象的数量大得多
- 谓词之间的语义关系极其复杂

例 (谓词锁)

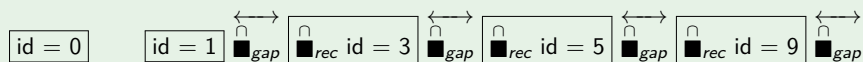
事务 T_1	事务 T_2	调度器的动作
<code>LOCK('id > 1')</code>		将锁授予 T_1
<code>SELECT MAX(val) FROM t</code> <code>WHERE id > 1;</code>		
	<code>LOCK('id = 3')</code>	拒绝
<code>SELECT MAX(val) FROM t</code> <code>WHERE id > 1;</code>		
<code>UNLOCK('id > 1')</code>		释放锁
<code>COMMIT</code>		
	<code>INSERT INTO t</code> <code>VALUES (3, 300);</code> <code>UNLOCK('id = 3');</code> <code>COMMIT</code>	将锁授予 T_2

间隙锁 (Gap Lock)

通过主索引，不仅对所有满足查询谓词的元组，还对元组之间的间隙加锁

例 (间隙锁)

对 $id > 1$ 这个区间加锁



9.3 基于时间戳的并发控制

9.3.1 基于时间戳的并发控制的基本概念

时间戳 (Timestamp)

定义 (事务的时间戳)

DBMS 为每个事务 T_i 分配一个唯一且固定的数, 作为 T_i 的时间戳, 记作 $TS(T_i)$

- 时间戳单调递增: 分配得越晚, 时间戳越大

分配时间戳的时机

分配时间戳的时机取决于并发控制协议的设计

- 事务启动时分配: 基本时间戳定序协议 (Basic Timestamp Ordering, Basic T/O)
- 事务执行过程中分配: 乐观并发控制协议 (Optimistic Concurrency Control, OCC)

基于时间戳的并发控制协议

- 目标串行调度: 由时间戳确定的串行调度
- 实际调度: 冲突等价于目标串行调度 (因此是冲突可串行化调度)

基于时间戳的并发控制与基于锁的并发控制的区别

	基于锁的并发控制	基于时间戳的并发控制
读/写对象前是否请求加锁	是	不需要
是否有目标串行调度	无	有（由时间戳决定）
是否有可能中止事务	否	是

9.3.2 基本时间戳定序协议

Basic T/O 的基本思想

- 事务启动时分配时间戳
- 事务执行过程中对每个读/写操作进行时间戳检查
- 如果执行该操作导致实际调度无法冲突等价于目标串行调度，因此拒绝执行该操作

例 (Basic T/O 的基本思想)

目标串行调度			实际调度	
事务 T_1	事务 T_2		事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$		$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN			BEGIN	
	BEGIN	$\neq$		BEGIN
$r(X)$			$r(X)$	$w(X)$
	$w(X)$			
读写冲突 $r_1(X), w_2(X)$			写读冲突 $w_2(X), r_1(X)$	

数据对象的时间戳

- 每个数据对象 X 记录 2 个时间戳
- $RTS(X)$: 成功读 X 的最晚的事务的时间戳（成功读 X 的事务的最大时间戳）
 - $WTS(X)$: 成功写 X 的最晚的事务的时间戳（成功写 X 的事务的最大时间戳）

Basic T/O 下的读操作（情况 1：过时读）

事务 T_i 读对象 X （过时读）

如果 $TS(T_i) < WTS(X)$ ，则

- ① 中止 T_i
- ② 重启 T_i （给 T_i 分配新的时间戳）

例（Basic T/O 下的过时读）

目标串行调度		实际调度	
事务 T_1	事务 T_2	事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$	$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN		BEGIN	
$r(X)$	BEGIN		BEGIN
	$w(X)$	$r(X)$ （此时 $WTS(X) = 2$ ）	$w(X)$
读写冲突 $r_1(X), w_2(X)$		读写冲突 $w_2(X), r_1(X)$	

Basic T/O 下的读操作（情况 2：正常读）

事务 T_i 读对象 X （正常读）

如果 $TS(T_i) \geq WTS(X)$ ，则

- ① 允许 T_i 读 X
- ② 将 $RTS(X)$ 更新为 $\max(RTS(X), TS(T_i))$
- ③ 在 T_i 的工作区（Workspace）创建 X 的局部副本，以保证可重复读

Basic T/O 下的写操作（情况 1：过时写 1）

事务 T_i 写对象 X （过时写 1）

如果 $TS(T_i) < RTS(X)$ ，则

- ① 中止 T_i
- ② 重启 T_i （给 T_i 分配新的时间戳）

例（Basic T/O 下的过时写 1）

时间戳确定的串行调度		实际的调度	
事务 T_1	事务 T_2	事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$	$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN		BEGIN	
$w(X)$	BEGIN		BEGIN
	$r(X)$	$w(X)$ （此时 $RTS(X) = 2$ ）	$r(X)$
读写冲突 $w_1(X), r_2(X)$		读写冲突 $r_2(X), w_1(X)$	

Basic T/O 下的写操作（情况 2：过时写 2）

事务 T_i 写对象 X （过时写 2）

如果 $TS(T_i) < WTS(X)$ ，则

- ① 中止 T_i
- ② 重启 T_i （给 T_i 分配新的时间戳）

例（Basic T/O 下的过时写 2）

目标串行调度		实际调度	
事务 T_1	事务 T_2	事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$	$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN		BEGIN	
$w(X)$	BEGIN		BEGIN
	$w(X)$	$w(X)$ （此时 $WTS(X) = 2$ ）	$w(X)$
读写冲突 $w_1(X), w_2(X)$		读写冲突 $w_2(X), w_1(X)$	

Basic T/O 下的写操作（情况 3：正常写）

事务 T_i 写对象 X （正常写）

如果 $TS(T_i) \geq RTS(X)$ 且 $TS(T_i) \geq WTS(X)$ ，则

- 1 允许 T_i 写 X
- 2 将 $WTS(X)$ 更新为 $\max(WTS(X), TS(T_i))$
- 3 在 T_i 的工作区中创建 X 的局部副本，以保证可重复读

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(B)$	
	BEGIN
	$r(B)$
	$w(B)$
$r(A)$	
	$r(A)$
	$w(A)$
COMMIT	COMMIT

对象	RTS	WTS
A	0	0
B	0	0

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(B)$	
	BEGIN
	$r(B)$
	$w(B)$
$r(A)$	
	$r(A)$
	$w(A)$
COMMIT	COMMIT

对象	RTS	WTS
A	0	0
B	1	0

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(B)$	
	BEGIN
	$r(B)$
	$w(B)$
$r(A)$	
	$r(A)$
	$w(A)$
COMMIT	COMMIT

对象	RTS	WTS
A	0	0
B	1	0

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	BEGIN
	r(B)
	w(B)
r(A)	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	0	0
B	2	0

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	BEGIN
	r(B)
	w(B)
r(A)	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	0	0
B	2	0

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	BEGIN
	r(B)
	w(B)
r(A)	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	0	0
B	2	2

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	BEGIN
	r(B)
	w(B)
r(A)	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	0	0
B	2	2

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	1	0
B	2	2

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	1	0
B	2	2

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	2	0
B	2	2

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	2	0
B	2	2

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	2	2
B	2	2

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	2	2
B	2	2

实际调度 $\equiv$ 目标串行调度

Basic T/O 示例 1

例 (Basic T/O 示例 1)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(B)	
	BEGIN
	r(B)
	w(B)
r(A)	
	r(A)
	w(A)
COMMIT	COMMIT

对象	RTS	WTS
A	2	2
B	2	2

实际调度是冲突可串行化调度吗?

Basic T/O 示例 2

例 (Basic T/O 示例 2)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
r(A)	
	BEGIN
	w(A)
	COMMIT
w(A)	
r(A)	
COMMIT	

对象	RTS	WTS
A	0	0
B	0	0

Basic T/O 示例 2

例 (Basic T/O 示例 2)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

对象	RTS	WTS
A	1	0
B	0	0

Basic T/O 示例 2

例 (Basic T/O 示例 2)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

对象	RTS	WTS
A	1	0
B	0	0

Basic T/O 示例 2

例 (Basic T/O 示例 2)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

对象	RTS	WTS
A	1	2
B	0	0

Basic T/O 示例 2

例 (Basic T/O 示例 2)

实际调度	
事务 T_1	事务 T_2
$TS(T_1) = 1$	$TS(T_2) = 2$
BEGIN	
$r(A)$	
	BEGIN
	$w(A)$
	COMMIT
$w(A)$	
$r(A)$	
COMMIT	

对象	RTS	WTS
A	1	2
B	0	0

Basic T/O 示例 2

例 (Basic T/O 示例 2)

实际调度		对象	RTS	WTS
事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$			
BEGIN r(A)		A	1	2
	BEGIN w(A)	B	0	0
w(A) 过时写, 中止 r(A) COMMIT	COMMIT			

实际调度 $\neq$ 目标串行调度

为过时写申辩

例 (过时写)

实际调度		设想的调度	
事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$	事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$
BEGIN r(A)		BEGIN r(A)	
	BEGIN w(A)	w(A) 假装在这里执行	BEGIN
w(A) COMMIT	COMMIT	COMMIT	w(A) COMMIT

- T_1 就是写 A 晚了一点点, 才被 T_2 先写了 A, 就不能 T_1 给个机会吗?
- 好吧, 鉴于 T_1 现在还没把事情搞大 (覆盖 T_2 写的 A), 我就给 T_1 一次机会: 只要 T_1 不执行这个过时写 (否则就真违反冲突可串行化了), 这次我就不追究了; 但丑话说在前面, 如果 T_1 后面再“出问题” (违反冲突可串行化), 它就必须中止

Thomas 写规则 (Thomas Write Rule)

事务 T_i 写对象 X (过时写 2)

如果 $TS(T_i) < WTS(X)$, 则忽略这个写操作

例 (Basic T/O 下的过时写 2)

目标串行调度		事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$
事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$		
BEGIN	BEGIN	BEGIN	BEGIN
w(X)	w(X)	w(X) (此时 $WTS(X) = 2$)	w(X)
写读冲突 $w_1(X), w_2(X)$		忽略 $w_2(X)$	

Thomas 写规则 (Thomas Write Rule)

例 (Thomas 写规则)

调度		Database
事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$	
BEGIN r(A)	BEGIN w(A)	对象
w(A) 过时写, 忽略 r(A) COMMIT	COMMIT	RTS
		WTS
		A
		B

Thomas 写规则 (Thomas Write Rule)

例 (Thomas 写规则)

调度		Database
事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$	
BEGIN $r(A)$	BEGIN $w(A)$	对象 RTS WTS
$w(A)$ 过时写, 忽略 $r(A)$ 过时读, 中止 COMMIT	COMMIT	A 1 2 B 0 0

Basic T/O 的性质

- 如果不使用 Thomas 写规则, 则 Basic T/O 一定产生冲突可串行化调度
- 因为任何事务都无需等待, 所以无死锁

Basic T/O 的不足

- 必须将对象复制到事务的工作区, 且频繁更新时间戳, 开销非常大
- 如果某个长事务总是和很多短事务发生冲突, 则这个长事务可能会“饿死” (不断中止并重启)
- 可能产生不可恢复的调度 (Unrecoverable Schedule)

不可恢复调度 (Unrecoverable Schedule)

Basic T/O 可生成下面的调度

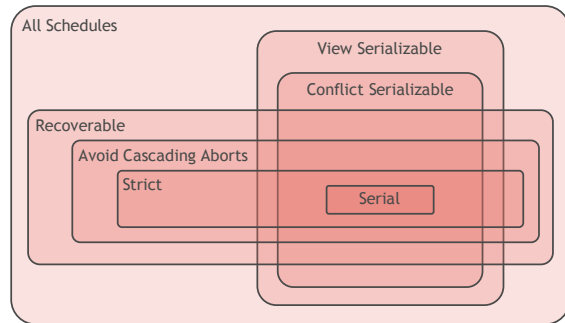
- 因为 T_2 读了未提交事务 T_1 修改过的 A , 所以必须撤销 T_2
- 然而, T_2 已提交。根据事务的持久性, 无法撤销 T_2

调度	
事务 T_1 $TS(T_1) = 1$	事务 T_2 $TS(T_2) = 2$
BEGIN $w(A)$	BEGIN $r(A)$ $w(B)$ COMMIT
ABORT	

可恢复调度 (Recoverable Schedule)

定义 (可恢复调度)

A schedule is recoverable if every transaction in that schedule commits only after all the transactions whose changes it reads have committed



9.3.2 乐观并发控制协议

乐观并发控制 (Optimistic Concurrency Control, OCC) 的基本思想

- 避免在执行每个操作前都检查操作的合法性，让事务的执行尽量不受干扰
- 事务的操作全部执行完后，再统一进行一次验证

乐观并发控制 (Optimistic Concurrency Control, OCC) 的适用场景

- 并发事务的冲突很少
- 大多数事务都很短

OCC 下事务的三个执行阶段

- 1 读阶段 (Read Phase)
- 2 验证阶段 (Validation Phase)
- 3 写阶段 (Write Phase)

读阶段 (Read Phase)

- DBMS 为事务 T 创建私有工作区 (Private Workspace)
- 事务 T 读的每个对象都要复制到工作区, 以保证可重复读
- 事务 T 对对象的修改写到工作区, 而不是数据库
- 将 T 的读和写分别记录到读集合 (Read Set) $RS(T)$ 和写集合 (Write Set) $WS(T)$ 中

例 (读阶段)

实际调度	
事务 T_1	事务 T_2
BEGIN	BEGIN
//读阶段	//读阶段
$r(A)$	$r(A)$
	//验证阶段
	//写阶段
	COMMIT
$w(A)$	
//验证阶段	
//写阶段	
COMMIT	

验证阶段 (Validation Phase)

- 验证阶段开始时, DBMS 为事务 T 分配时间戳
- DBMS 检查 T 的写集合 $WS(T)$, 判断事务 T 是否与其他活跃事务冲突, 即是否造成实际的调度不冲突等价于由时间戳确定的目标串行调度
- 如果通过验证, 则进入写阶段; 否则, 中止 T , 重启 T 为新事务

例 (验证阶段)

实际调度	
事务 T_1	事务 T_2
BEGIN	BEGIN
//读阶段	//读阶段
$r(A)$	$r(A)$
	//验证阶段, 分配时间戳 $TS(T_2) = 1$
	冲突等价于串行调度 (T_2)
	//写阶段
	COMMIT
$w(A)$	
//验证阶段, 分配时间戳 $TS(T_1) = 2$	
冲突等价于串行调度 (T_2, T_1)	
//写阶段	
COMMIT	

写阶段 (Write Phase)

- 将 T 的写集合 $WS(T)$ 中的对象写回数据库

例 (写阶段)

实际调度	
事务 T_1	事务 T_2
BEGIN	BEGIN
//读阶段	//读阶段
$r(A)$	$r(A)$
	//验证阶段, 分配时间戳 $TS(T_2) = 1$
	//写阶段
	COMMIT
$w(A)$	
//验证阶段, 分配时间戳 $TS(T_1) = 2$	
//写阶段	
COMMIT	

OCC 示例

例 (OCC 示例)

实际调度	
事务 T_1	事务 T_2
BEGIN	BEGIN
//Read	
$r(A)$	//Read
	$r(A)$
	//Validate
	//Write
	COMMIT
$w(A)$	
//Validate	
//Write	
COMMIT	

数据库		
对象	值	WTS
A	123	0

OCC 示例

例 (OCC 示例)

实际调度	
事务 T_1	事务 T_2
BEGIN	BEGIN
//Read	
$r(A)$	//Read
	$r(A)$
	//Validate
	//Write
	COMMIT
$w(A)$	
//Validate	
//Write	
COMMIT	

数据库		
对象	值	WTS
A	123	0

T_1 的工作区		
对象	值	WTS

OCC 示例

例 (OCC 示例)

实际调度

事务 T_1	事务 T_2
BEGIN	BEGIN
//Read	
$r(A)$	//Read
	$r(A)$
	//Validate
	//Write
	COMMIT
$w(A)$	
//Validate	
//Write	
COMMIT	

数据库

对象	值	WTS
A	123	0

T_1 的工作区

对象	值	WTS
A	123	0

OCC 示例

例 (OCC 示例)

实际调度	
事务 T_1	事务 T_2
BEGIN	BEGIN
//Read $r(A)$	//Read $r(A)$
	//Validate
	//Write
	COMMIT
$w(A)$	
//Validate	
//Write	
COMMIT	

数据库		
对象	值	WTS
A	123	0

T_1 的工作区		
对象	值	WTS
A	123	0

T_2 的工作区		
对象	值	WTS

OCC 示例

例 (OCC 示例)

实际调度		数据库		
事务 T_1	事务 T_2	对象	值	WTS
BEGIN	BEGIN	A	123	0
//Read $r(A)$	//Read $r(A)$	T_1 的工作区		
	//Validate	对象	值	WTS
	//Write	A	123	0
	COMMIT	T_2 的工作区		
$w(A)$		对象	值	WTS
//Validate		A	123	0
//Write				
COMMIT				

OCC 示例

例 (OCC 示例)

实际调度		数据库		
事务 T_1	事务 T_2	对象	值	WTS
BEGIN	BEGIN	A	123	0
//Read $r(A)$	//Read $r(A)$	T_1 的工作区		
	//Validate	对象	值	WTS
	$TS(T_2) = 1$	A	123	0
	//Write	T_2 的工作区		
$w(A)$		对象	值	WTS
//Validate		A	123	0
//Write				
COMMIT	COMMIT			

OCC 示例

例 (OCC 示例)

实际调度		数据库		
事务 T_1	事务 T_2	对象	值	WTS
BEGIN	BEGIN	A	123	0
//Read $r(A)$	//Read $r(A)$	T_1 的工作区		
	//Validate	对象	值	WTS
	$TS(T_2) = 1$	A	123	0
	//Write	T_2 的工作区		
	COMMIT	对象	值	WTS
$w(A)$		A	123	0
//Validate				
//Write				
COMMIT				

OCC 示例

例 (OCC 示例)

实际调度		数据库		
事务 T_1	事务 T_2	对象	值	WTS
BEGIN	BEGIN	A	123	0
//Read $r(A)$	//Read $r(A)$	T_1 的工作区		
	//Validate	对象	值	WTS
	$TS(T_2) = 1$	A	123	0
	//Write			
$w(A)$				
//Validate				
//Write				
COMMIT	COMMIT			

OCC 示例

例 (OCC 示例)

实际调度		
事务 T_1	事务 T_2	
BEGIN	BEGIN	
//Read $r(A)$	//Read $r(A)$	
	//Validate $TS(T_2) = 1$	
	//Write COMMIT	
$w(A)$		
//Validate		
//Write COMMIT		

数据库		
对象	值	WTS
A	123	0

T_1 的工作区		
对象	值	WTS
A	456	∞

OCC 示例

例 (OCC 示例)

实际调度		
事务 T_1	事务 T_2	
BEGIN	BEGIN	
//Read $r(A)$	//Read $r(A)$	
	//Validate $TS(T_2) = 1$	
	//Write COMMIT	
$w(A)$		
//Validate $TS(T_1) = 2$		
//Write COMMIT		

数据库		
对象	值	WTS
A	123	0

T_1 的工作区		
对象	值	WTS
A	456	∞

OCC 示例

例 (OCC 示例)

实际调度		
事务 T_1	事务 T_2	
BEGIN	BEGIN	
//Read $r(A)$	//Read $r(A)$	
	//Validate $TS(T_2) = 1$	
	//Write COMMIT	
$w(A)$		
//Validate $TS(T_1) = 2$		
//Write COMMIT		

数据库		
对象	值	WTS
A	456	2

T_1 的工作区		
对象	值	WTS
A	456	∞

OCC 示例

例 (OCC 示例)

实际调度		
事务 T_1	事务 T_2	
BEGIN	BEGIN	
//Read $r(A)$	//Read $r(A)$	
	//Validate $TS(T_2) = 1$	
	//Write COMMIT	
$w(A)$		
//Validate $TS(T_1) = 2$		
//Write COMMIT		

数据库		
对象	值	WTS
A	456	2

验证阶段 (Validation Phase)

事务 T_j 进入验证阶段后

DBMS 为 T_j 分配时间戳 $TS(T_j)$

对任意 $TS(T_i) < TS(T_j)$ 的事务 T_i , 如果下面 3 个条件之一满足, 则 T_j 与 T_i 不冲突

- 条件 1: 在 T_j 开始前, T_i 已完成 3 个阶段
- 条件 2: 在 T_j 的写阶段开始前, T_i 已完成 3 个阶段, 且 T_i 写的对象均未被 T_j 读过, 即 $WS(T_i) \cap RS(T_j) = \emptyset$
- 条件 3: 在 T_j 的读阶段完成前, T_i 的读阶段已完成, 且 T_i 写的对象均未被 T_j 读写过, 即 $WS(T_i) \cap RS(T_j) = \emptyset$ 且 $WS(T_i) \cap WS(T_j) = \emptyset$
- 每个条件都保证了 T_j 的写对 T_i 不可见, T_i 的写对 T_j 也不可可见

如果所有满足 $TS(T_i) < TS(T_j)$ 的事务 T_i 与 T_j 都不冲突, 则 T_j 通过验证, 进入写阶段

验证条件 1

验证条件 1

在 T_j 开始前, T_i 已完成三个阶段

T_i
读阶段 R(X)
验证阶段
写阶段 W(X)

	目标串行调度	实际调度
RW 冲突	$r_i(X), w_j(X)$	$r_i(X), w_j(X)$
WR 冲突	$w_i(X), r_j(X)$	$w_i(X), r_j(X)$
WW 冲突	$w_i(X), w_j(X)$	$w_i(X), w_j(X)$

T_j
读阶段 R(X)
验证阶段
写阶段 W(X)

验证条件 2

验证条件 2

- 在 T_j 的写阶段开始前, T_i 已完成三个阶段
- $WS(T_i) \cap RS(T_j) = \emptyset$

T_i
读阶段 R(X)
验证阶段
写阶段 W(X)

	目标串行调度	实际调度
RW 冲突	$r_i(X), w_j(X)$	$r_i(X), w_j(X)$
WW 冲突	$w_i(X), w_j(X)$	$w_i(X), w_j(X)$
WR 冲突	无	无

- 时间戳确定的 WR 冲突可能交换执行顺序, 但因为 $WS(T_i) \cap RS(T_j) = \emptyset$, 所以实际无 WR 冲突

T_j
读阶段 R(Y)
验证阶段
写阶段 W(X)

验证条件 2

例 (满足验证条件 2 的调度)

调度	
T_1	T_2
BEGIN	BEGIN
	//Read
	$r(A)$
//Read	$w(A)$
$r(A)$	
//Validate	
//Write	
COMMIT	
	//Validate
	//Write
	COMMIT

- $W(T_1) \cap R(T_2) = \emptyset$
- 目标串行调度: $r_1(A), r_2(A), w_2(A)$
- 实际调度: $r_2(A), r_1(A), w_2(A)$
- 二者冲突等价

验证条件 2

例 (不满足验证条件 2 的调度)

调度	
T_1	T_2
BEGIN	BEGIN
//Read $r(A)$ $w(A)$	//Read $r(A)$
//Validate	
//Write COMMIT	
	//Validate
	//Write COMMIT

- $W(T_1) \cap R(T_2) = \{A\} \neq \emptyset$
- 目标串行调度: $r_1(A), w_1(A), r_2(A)$
- 实际调度: $r_1(A), r_2(A), w_1(A)$
- 二者不冲突等价

验证条件 3

验证条件 3

- 1 在 T_j 的读阶段完成前, T_i 的读阶段已完成
- 2 $WS(T_i) \cap RS(T_j) = \emptyset$
- 3 $WS(T_i) \cap WS(T_j) = \emptyset$

		目标串行调度	实际调度
		$r_i(X), w_j(X)$	$r_i(X), w_j(X)$
T_i 读阶段 $R(X)$ 验证阶段 写阶段 $W(Z)$	T_j 读阶段 $R(Y)$ 验证阶段 写阶段 $W(X)$	RW 冲突	无
		WW 冲突	无
		WR 冲突	无

- 时间戳确定的 WR 冲突可能交换执行顺序, 但因为 $WS(T_i) \cap WS(T_j) = \emptyset$, 所以实际无 WW 冲突
- 时间戳确定的 WR 冲突可能交换执行顺序, 但因为 $WS(T_i) \cap RS(T_j) = \emptyset$, 所以实际无 WR 冲突

验证条件 3

例 (满足验证条件 3 的调度)

调度	
T_1	T_2
BEGIN	BEGIN
//Read $r(A)$ $w(A)$	//Read $r(B)$
//Validate	
	$w(B)$
	//Validate
	//Write COMMIT
//Write COMMIT	

- $W(T_1) \cap R(T_2) = \emptyset$
- $W(T_1) \cap W(T_2) = \emptyset$
- 目标串行调度: $r_1(A), w_1(A), r_2(B), w_2(B)$
- 实际调度: $r_1(A), r_2(B), w_2(B), w_1(A)$
- 二者冲突等价

验证条件 3

例 (不满足验证条件 3 的调度)

调度	
T_1	T_2
BEGIN	BEGIN
//Read $r(A)$ $w(A)$	//Read $r(A)$
//Validate	
	$r(B)$
	//Validate
	//Write COMMIT
//Write COMMIT	

- $W(T_1) \cap R(T_2) = \{A\} \neq \emptyset$
- $W(T_1) \cap W(T_2) = \emptyset$
- 目标串行调度: $r_1(A), w_1(A), r_2(A), w_2(B)$
- 实际调度: $r_1(A), r_2(A), r_2(B), w_1(A)$
- 二者冲突等价

串行验证 (Serial Validation)

在临界区 (Critical Section) 中内执行事务的验证阶段和写阶段

- 任何时刻只有一个事务处于验证/写阶段
- 当一个事务正在验证时，其他事务不得提交
- 如果一个事务已经完成验证阶段，则只有在其写阶段完成后，其他事务才能开始验证

OCC 的优点

在并发事务冲突很少的情况下，OCC 很有效

- 事务都是只读的
- 事务访问的数据子集互不相交

OCC 的缺点

- 将数据复制到工作区的开销很大
- 验证阶段和写阶段是瓶颈
- OCC 比 2PL 的事务中止代价高，因为中止事务的读阶段已经完成

Basic T/O 和 OCC 的区别

	Basic T/O	OCC
分配时间戳	事务启动时	验证阶段
合法性检查	每个读写操作前	事务验证阶段
事务运行时代价	高	低
中止事务浪费的计算	低	高

9.4 多版本并发控制

9.4.1 多版本并发控制的基本概念

多版本并发控制 (Multi-Version Concurrency Control, MVCC)

让每个事务仅在自己启动时可见的数据库快照 (Snapshot) 上执行，从而达到如下目的：

- 读不阻塞写
- 写不阻塞读

MVCC 不仅仅是一种并发控制协议，它还改变了数据库存储的设计，是一种新的 DBMS 设计方法

版本 (Version)

DBMS 为数据库中每个 (逻辑) 对象维护多个 (物理) 版本

- 当事务写一个对象时，DBMS 为该对象创建一个新的版本
- 当事务读一个对象时，它读的是该事务启动时已存在的由已提交事务写入的最新版本

MVCC 下事务的时间戳

定义 (提交时间戳)

事务 T 提交时, 分配给它的时间戳, 记作 $CommitTS(T)$

- 互不相同
- 单调递增
- 唯一确定了并发事务的等价串行调度

定义 (读时间戳)

事务 T 启动时, 最后提交的事务的提交时间戳, 记作 $ReadTS(T)$

- 不同事务的读时间戳可以相同
- 决定了事务可见的元组版本

元组版本的组成

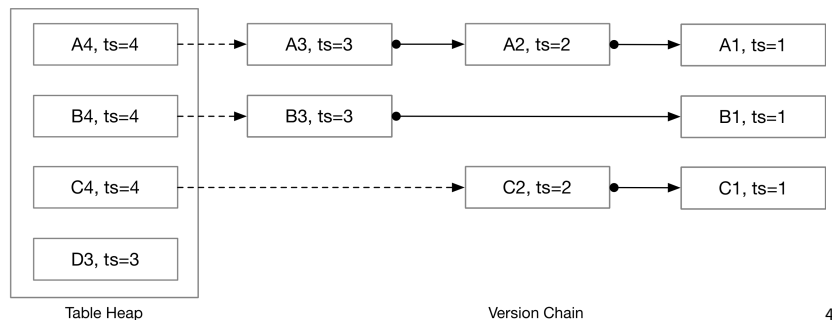
元组的每个版本存储以下数据

- 元组 rid
- 版本数据
- beginTS: 创建该版本的事务的提交时间戳
- endTS: 终结该版本的事务的提交时间戳 (该事务创建了下一个版本或删除了该元组)

例 (元组版本)

Rid	Value	Begin	End
(0, 0)	123	0	2
(0, 0)	456	2	5
(0, 0)	789	5	∞
(0, 1)	abc	0	3
(0, 1)	def	3	∞

元组版本的组成



这里的版本都是已提交的版本, ts 代表版本的 beginTS

4

思考题

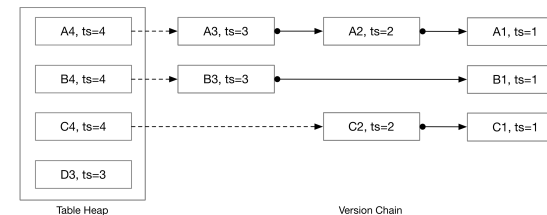
读时间戳是 2 的事务启动时看到的已提交版本有哪些?

⁴<https://15445.courses.cs.cmu.edu/spring2025/project4/img/1-1-ts.png>

版本可见性

定理 (版本可见性)

元组 t 的已提交版本 t_i 对事务 T 可见, 当且仅当 $beginTS(t_i)$ 是 t 的所有版本的 $beginTS$ 中满足小于等于 T 的读时间戳 $ReadTS(T)$ 的最大值



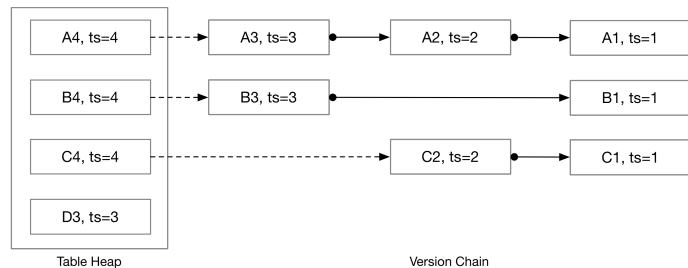
例 (版本可见性)

读时间戳是 2 的事务现在能见到的已提交版本有 A2、B1 和 C2

一致性快照 (Consistent Snapshot)

定义 (一致性快照)

事务 T 启动时全部可见的已提交元组版本的集合



例 (一致性快照)

读时间戳是 2 的事务的一致性快照是 $\{A2, B1, C2\}$

快照隔离 (Snapshot Isolation, SI)

定义 (快照隔离)

事务 T 的读操作只在 T 的一致性快照上进行

快照隔离的特点

- 写事务不阻塞读事务
- 读事务不阻塞写事务
- 只读事务直接在一致性快照上读，无需加锁

未提交事务创建的版本

定理 (未提交事务创建的版本的可性)

未提交事务创建的版本仅对自己可见

如何表示一个版本是哪个未提交事务创建的?

- 未提交事务还没有提交时间戳
- 将版本号置为临时时间戳 ($1LL \ll 62$) + txn_id (64 位整数), 其中 txn_id 为事务 ID

思考

为什么临时时间戳不是 $(1LL \ll 63) + txn_id$?

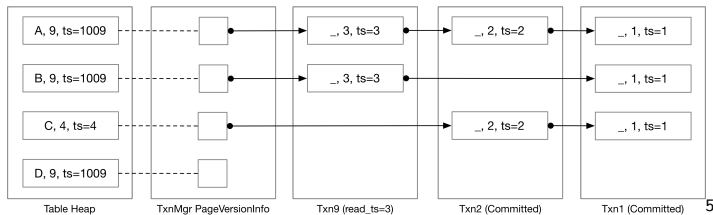
9.4.2 多版本并发控制下的操作

读元组 (情况 1)

事务 T 读一个元组

读元组的方法 (情况 1)

如果 t 的最新版本是已提交的, 且是对 T 可见的, 则直接读 t 的最新版本



例 (读元组 (情况 1))

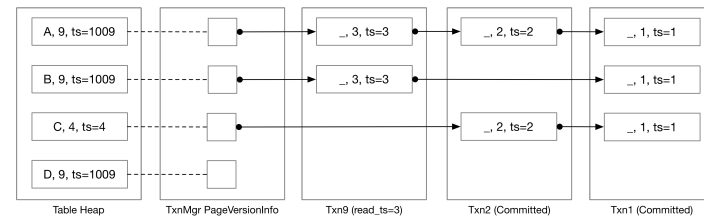
读时间戳是 5 的事务读元组 C 的版本是 (C, 4), ts=4

⁵<https://15445.courses.cs.cmu.edu/spring2025/project4/img/2-3-seqscan.png>

读元组 (情况 2)

读元组的方法 (情况 2)

如果 t 的最新版本是已提交的, 但比 T 的读时间戳还要新, 则需要在 t 的最新版本的基础上重建 t 可见的版本



例 (读元组 (情况 2))

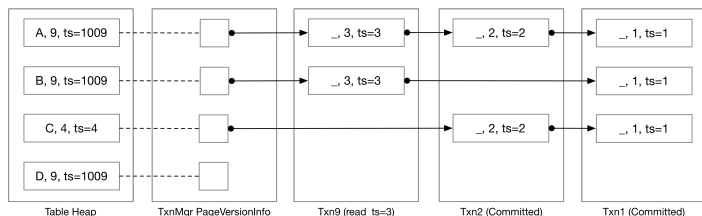
读时间戳是 3 的事务读元组 C 的版本是 (C, 2), ts=2

<https://15445.courses.cs.cmu.edu/spring2025/project4/img/2-3-seqscan.png>

读元组 (情况 3)

读元组的方法 (情况 3)

如果 t 的最新版本是未提交的, 并且是 T 自己创建的, 则直接读 t 的最新版本



例 (读元组 (情况 3))

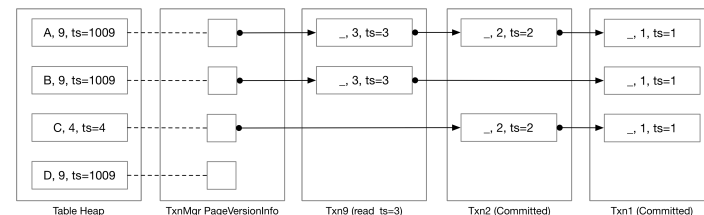
事务 9 读元组 A 的版本是 (A, 9), ts=1009

注: 1009 代表 $(1LL \ll 62) + 9$

读元组 (情况 4)

读元组的方法 (情况 4)

如果 t 的最新版本是未提交的, 且不是 T 创建的, 则需要在 t 的最新版本的基础上重建 t 可见的版本



例 (读元组 (情况 4))

读时间戳是 3 的事务 10 读元组 A 的版本是 (A, 3), ts=3

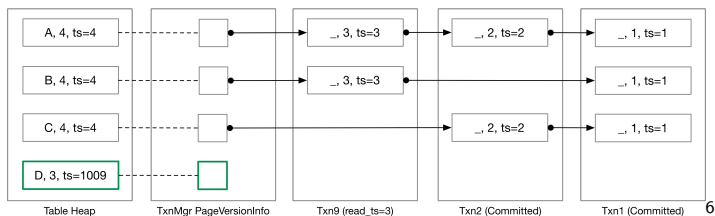
<https://15445.courses.cs.cmu.edu/spring2025/project4/img/2-3-seqscan.png>

插入元组

事务 T 插入元组 t (插入元组不会产生写写冲突, 因为该元组之前并不存在)

插入元组的方法

- 1 生成 T 的临时时间戳 ($1LL \ll 62$) + $tid(T)$
- 2 创建 t 的最新版本, 其 $beginTS$ 置为 T 的临时时间戳



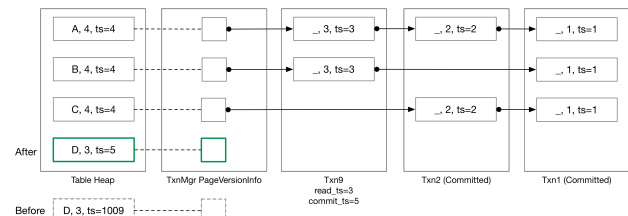
例 (插入元组)

事务 9 插入元组 (D, 3), ts=1009

事务提交

提交事务的方法

- 1 获取事务 T 的提交时间戳 $CommitTS(T)$
- 2 将 T 创建的元组版本中的临时时间戳替换为 $CommitTS(T)$
- 3 将 T 的状态置为“提交”



例 (事务提交)

事务 9 提交, 设事务 9 获得的提交时间戳为 5

修改元组

事务 T 修改元组 t 可能产生写写冲突

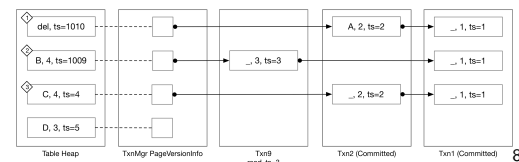
修改元组的方法

- 1 如果事务 T 修改元组 t 产生写写冲突, 则中止 T , 重启 T 为新事务
- 2 否则, 事务 T 创建元组 t 的新版本

写写冲突检测

写写冲突检测方法 (First Writer Wins)

如果其他事务已经创建了元组 t 的一个版本 (无论是否提交) 比事务 T 的可见版本还新, 则 T 修改元组 t 会产生写写冲突



例 (写写冲突)

- 事务 10 (读时间戳是 3) 修改元组 B: 写写冲突!
- 事务 9 (读时间戳是 3) 修改元组 C: 写写冲突!

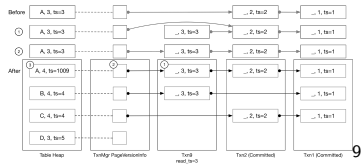
⁸<https://15445.courses.cs.cmu.edu/spring2025/project4/img/3-4-write-conflict.png>

创建新版本

创建新版本的方法

前提条件：不存在写写冲突

- 1 生成事务 T 的临时时间戳 ($1LL \ll 62$) + $tid(T)$
- 2 基于 T 可见的 t 版本，创建一个新版本，将其 beginTS 置为 T 的临时时间戳



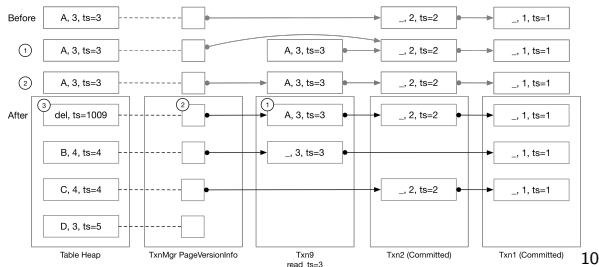
例 (修改元组)

事务 9 (读时间戳是 3) 将元组 A 修改为 (A, 4)

⁹<https://15445.courses.cs.cmu.edu/spring2025/project4/img/3-4-update.png>

删除元组

删除元组与修改元组的过程类似，只是版本数据用“墓碑”表示即可



例 (修改元组)

事务 9 (读时间戳是 3) 删除元组 (A, 3)

¹⁰<https://15445.courses.cs.cmu.edu/spring2025/project4/img/3-4-delete.png>

SI 下的 MVCC 示例 1

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	-
$r(A)$					
	BEGIN				
	$w(A)$				
$r(A)$					
COMMIT					
	COMMIT				

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active

SI 下的 MVCC 示例 1

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	-
$r(A) \leftarrow A_0$					
	BEGIN				
	$w(A)$				
$r(A)$					
COMMIT					
	COMMIT				

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active

SI 下的 MVCC 示例 1

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN	BEGIN $w(A)$ COMMIT	A_0	123	0	-
$r(A)$					
$r(A)$					
COMMIT					

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Active

SI 下的 MVCC 示例 1

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN	BEGIN $w(A)$ COMMIT	A_0	123	0	1002
$r(A)$		A_1	456	1002	∞
$r(A)$					
COMMIT					

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Active

SI 下的 MVCC 示例 1

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN	BEGIN $w(A)$ COMMIT	A_0	123	0	1002
$r(A)$		A_1	456	1002	∞
$r(A) \leftarrow A_0$					
COMMIT					

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Active

SI 下的 MVCC 示例 1

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN	BEGIN $w(A)$ COMMIT	A_0	123	0	2
$r(A)$		A_1	456	2	∞
$r(A)$					
COMMIT					

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	1	Committed
T_2	0	2	Committed

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	∞
$r(A)$					
$w(A)$	BEGIN				
	$r(A)$				
	$w(A)$				
$r(A)$					
COMMIT					
	COMMIT				

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	∞
$r(A)$					
$r(A) \leftarrow A_0$					
$w(A)$	BEGIN				
	$r(A)$				
	$w(A)$				
$r(A)$					
COMMIT					
	COMMIT				

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	1001
$r(A)$		A_1	456	1001	∞
$w(A)$	BEGIN				
	$r(A)$				
	$w(A)$				
$r(A)$					
COMMIT					
	COMMIT				

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	1001
$r(A)$		A_1	456	1001	∞
$w(A)$	BEGIN				
	$r(A)$				
	$w(A)$				
$r(A)$					
COMMIT					
	COMMIT				

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	1001
$r(A)$		A_1	456	1001	∞
$w(A)$	BEGIN				
	$r(A) \leftarrow A_0$				
	$w(A)$				
$r(A)$		事务状态表			
COMMIT		TxnId	ReadTS	CommitTS	Status
		T_1	0	—	Active
	COMMIT	T_2	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	1001
$r(A)$		A_1	456	1001	∞
$w(A)$	BEGIN				
	$r(A)$				
	$w(A)$ Stall...				
$r(A)$		事务状态表			
COMMIT		TxnId	ReadTS	CommitTS	Status
		T_1	0	—	Active
	COMMIT	T_2	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	1001
$r(A)$		A_1	456	1001	∞
$w(A)$	BEGIN				
	$r(A)$				
	$w(A)$ Stall...				
$r(A) \leftarrow A_1$		事务状态表			
COMMIT		TxnId	ReadTS	CommitTS	Status
		T_1	0	—	Active
	COMMIT	T_2	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	1
$r(A)$		A_1	456	1	∞
$w(A)$	BEGIN				
	$r(A)$				
	$w(A)$ Stall...				
$r(A)$		事务状态表			
COMMIT		TxnId	ReadTS	CommitTS	Status
		T_1	0	1	Committed
	COMMIT	T_2	0	—	Active

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	1002
$r(A)$		A_1	456	1	∞
$w(A)$		A_2	789	1002	∞
	BEGIN	事务状态表			
	$r(A)$	TxnId	ReadTS	CommitTS	Status
	$w(A)$	T_1	0	1	Committed
$r(A)$		T_2	0	—	Active
COMMIT	COMMIT				

SI 下的 MVCC 示例 2 (有写写冲突, 但无冲突检查)

不进行写写冲突检查会出现什么问题?

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	2
$r(A)$		A_1	456	1	∞
$w(A)$		A_2	789	2	∞
	BEGIN	事务状态表			
	$r(A)$	TxnId	ReadTS	CommitTS	Status
	$w(A)$	T_1	0	1	Committed
$r(A)$		T_2	0	2	Committed
COMMIT	COMMIT				

时间戳 2 以后同时存在 2 个版本

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	∞
$r(A)$					
$w(A)$					
	BEGIN	事务状态表			
	$r(A)$	TxnId	ReadTS	CommitTS	Status
	$w(A)$	T_1	0	—	Active
$r(A)$					
COMMIT	COMMIT				

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度		数据库			
事务 T_1	事务 T_2	Version	Value	Begin	End
BEGIN		A_0	123	0	∞
$r(A) \leftarrow A_0$					
$w(A)$					
	BEGIN	事务状态表			
	$r(A)$	TxnId	ReadTS	CommitTS	Status
	$w(A)$	T_1	0	—	Active
$r(A)$					
COMMIT	COMMIT				

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度	
事务 T_1	事务 T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A)
	w(A)
r(A)	
COMMIT	
	COMMIT

数据库			
Version	Value	Begin	End
A_0	123	0	1001
A_1	456	1001	∞

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度	
事务 T_1	事务 T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A)
	w(A)
r(A)	
COMMIT	
	COMMIT

数据库			
Version	Value	Begin	End
A_0	123	0	1001
A_1	456	1001	∞

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Active

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度	
事务 T_1	事务 T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A) $\leftarrow A_0$
	w(A)
r(A)	
COMMIT	
	COMMIT

数据库			
Version	Value	Begin	End
A_0	123	0	1001
A_1	456	1001	∞

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Active

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度	
事务 T_1	事务 T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A)
	w(A) Abort...
r(A)	
COMMIT	
	COMMIT

数据库			
Version	Value	Begin	End
A_0	123	0	1001
A_1	456	1001	∞

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Aborted

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度	
事务 T_1	事务 T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A)
	w(A) Abort...
$r(A) \leftarrow A_1$	
COMMIT	
	COMMIT

数据库			
Version	Value	Begin	End
A_0	123	0	1001
A_1	456	1001	∞

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	—	Active
T_2	0	—	Aborted

SI 下的 MVCC 示例 3 (有写写冲突检查)

进行写写冲突检查

例 (快照隔离下的 MVCC)

调度	
事务 T_1	事务 T_2
BEGIN	
r(A)	
w(A)	
	BEGIN
	r(A)
	w(A) Abort...
r(A)	
COMMIT	
	COMMIT

数据库			
Version	Value	Begin	End
A_0	123	0	1
A_1	456	1	∞

事务状态表			
TxnId	ReadTS	CommitTS	Status
T_1	0	1	Committed
T_2	0	—	Aborted

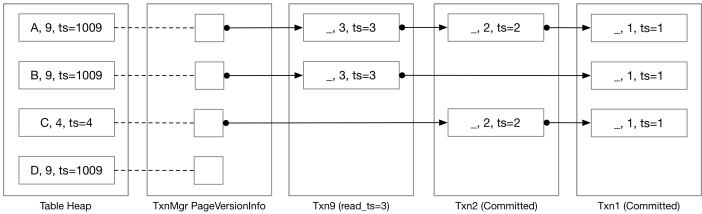
垃圾回收 (Garbage Collection)

- 版本越多，系统负担越重
- 所有版本都必须保留吗？

水位线 (Watermark)

定义 (水位线)
所有活跃事务 (即未提交，也未中止) 的最小读时间戳

- 水位线有什么用？
- 如何快速计算水位线？



例 (水位线)
如果水位线是 2，则 $ts = 1$ 的版本都可以删除

写偏斜异常 (Write Skew)

MVCC/SI 可能会导致非可串行化调度

例 (写偏斜异常)

id	on-duty	id	on-duty
1	true	1	false
2	true	2	false

- T1: 只要不是所有元组 on-duty 值都为 false, 就执行
| UPDATE T SET on-duty = FALSE WHERE id = 1;
- T2: 只要不是所有元组 on-duty 值都为 false, 就执行
| UPDATE T SET on-duty = FALSE WHERE id = 2;
- 任何串行调度都得不到上面的结果

探究式学习: MVCC/SI 的可串行化检查

通过 LLM 学习

- 如何验证 MVCC/SI 下生成的是否是可串行化调度

总结

- ① 9.1 并发控制
- ② 9.2 基于锁的并发控制
 - 9.2.1 锁
 - 9.2.2 两阶段封锁协议
 - 9.2.3 级联中止
 - 9.2.4 死锁
 - 9.2.5 多粒度锁
 - 9.2.6 幻读异常
- ③ 9.3 基于时间戳的并发控制
 - 9.3.1 基于时间戳的并发控制的基本概念
 - 9.3.2 基本时间戳定序协议
 - 9.3.2 乐观并发控制协议
- ④ 9.4 多版本并发控制
 - 9.4.1 多版本并发控制的基本概念
 - 9.4.2 多版本并发控制下的操作

致谢

2026 版以前

- 感谢田宇丰同学 (2022112431)、曹熠坤 (2022110790) 指出并修正课件中的错误